

文本、代码、图像和音频为四个模态，一个能够处理两者及以上的模型被称为多模态模型。

一、视觉变压器

基于 transformer 的模型成功地用于各种语言建模任务，从分类和聚类到搜索和生成建模。因此，研究人员一直在寻找一种方法，将 transformer 推广到计算机视觉领域。他们提出的方法被称为视觉转换器（ViT）。

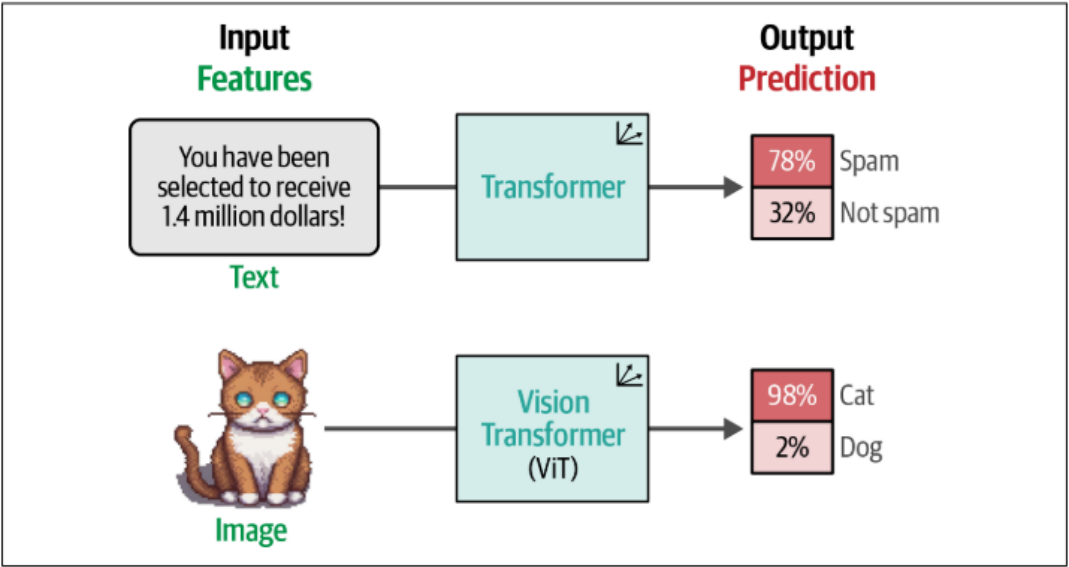


Figure 9-2. Both the original Transformer as well as the Vision Transformer take unstructured data, convert it to numerical representations, and finally use that for tasks like classification.

由于图像不是由单词组成的，因此 tokenization 不能用于视觉数据。ViT 的作者提出了一种将图像标记为“单词”的方法，这允许他们使用原始的编码器结构。假设有一只猫的图像，这个图像由 512×512 像素表示。每个单独的像素并不能传达太多的信息，但是当你把一块块像素组合在一起时，你会慢慢地看到更多的信息。将图像分成多个图像块后，转变为扁平化输入。

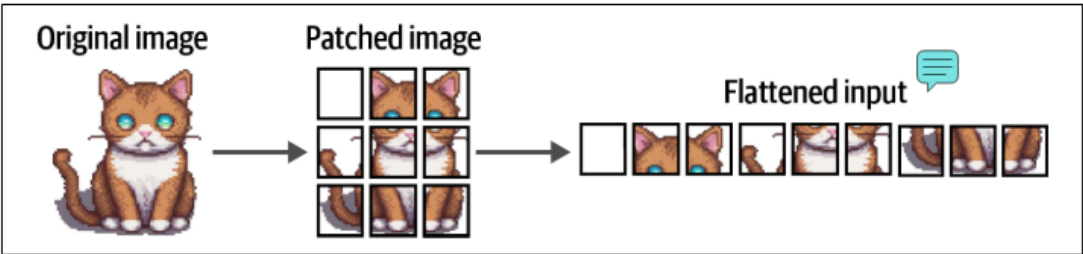


Figure 9-4. The “tokenization” process for image input. It converts an image into patches of subimages.

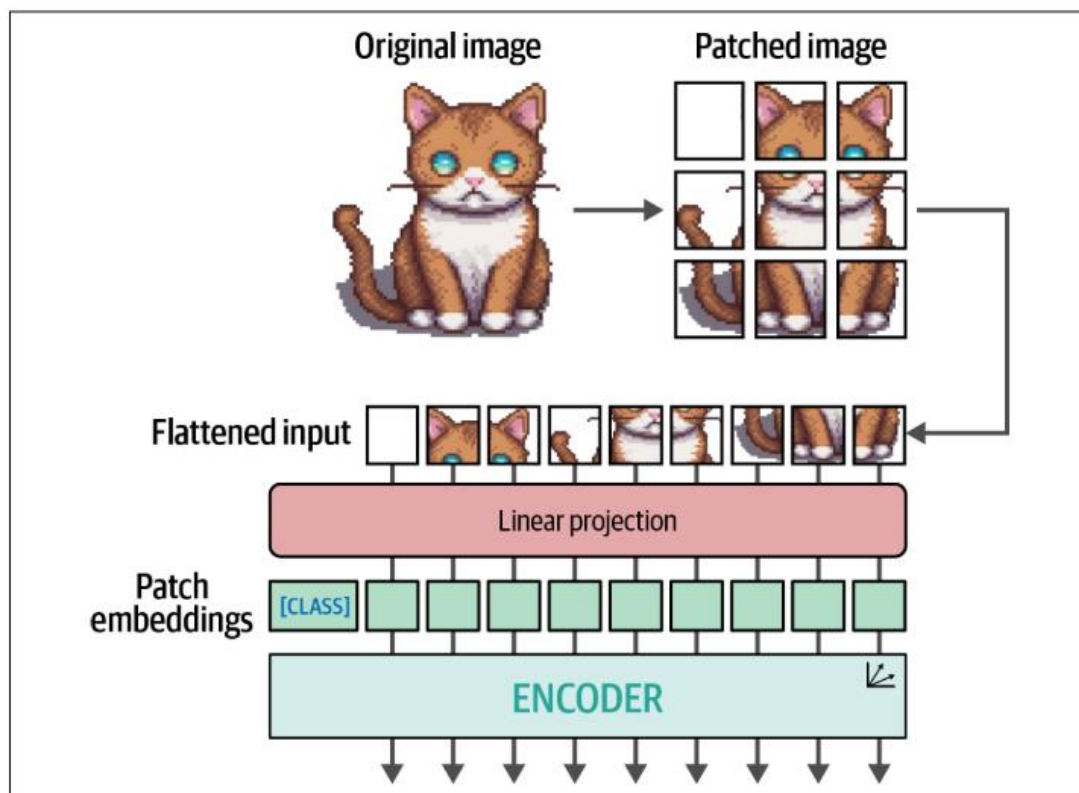
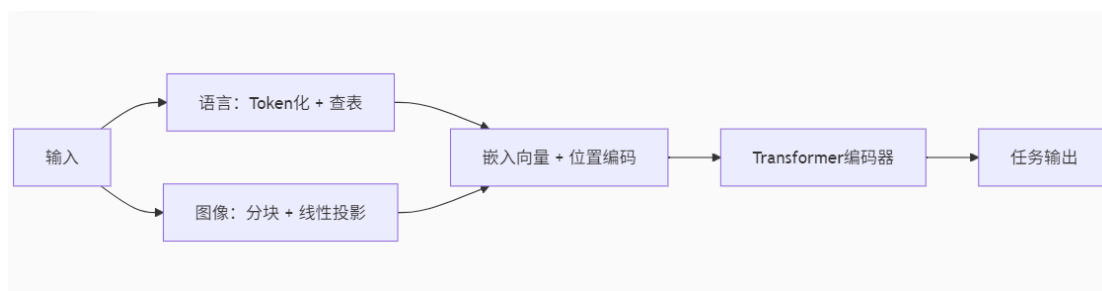


Figure 9-5. The main algorithm behind ViT. After patching the images and linearly projecting them, the patch embeddings are passed to the encoder and treated as if they were textual tokens.

经过线性投影转变为嵌入向量，从这时起，文本和图像的训练方式得到统一，后续的步骤都是相同的。也就是一旦生成嵌入向量，后续训练流程没有区别。



需要注意的是，ViT 没有 token 库，因为每个图像块都是不同的，很难找到两个相同的图像块。

嵌入向量更新的本质

1. 语言模型（如 BERT）中的 Token 更新

目标：捕获词义与上下文关系

例：句子 “银行在河边” → “银行”的嵌入需根据上下文更新，避免与金融“银行”混淆。

机制：

通过自注意力（Self-Attention）计算 Token 间的依赖关系。

更新后：每个 Token 嵌入包含**全局语义信息**（如指代河流还是金融机构）。

2. ViT 中的图像块嵌入更新

目标：构建视觉特征的层次化表示

例：猫的图片 → 浅层更新边缘/纹理，深层更新局部结构（耳朵、眼睛），最终整合为“猫”的语义。

机制：自注意力计算图像块间的空间与语义关系（如猫耳朵与猫眼睛的关联性）。

更新后：每个块嵌入从低级视觉特征（颜色、边缘）→ 高级语义特征（物体部件→完整物体）。

二、多模态嵌入模型

多模态嵌入模型可以将不同模态的数据嵌入到同一个向量空间，并且如果内容相同，则在向量空间中的距离非常接近。这样做的好处是可以比较多模态表示，例如，使用这种多模态嵌入模型，我们可以根据输入文本找到图像。

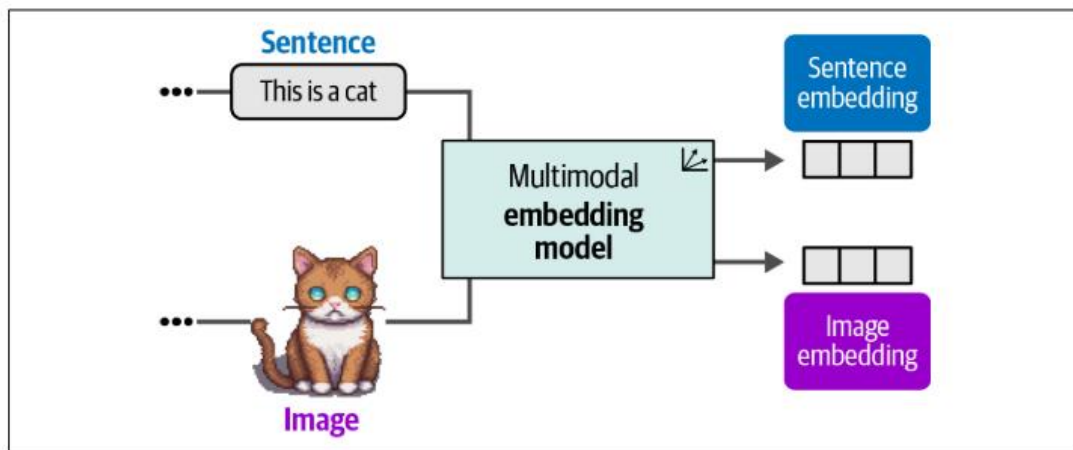


Figure 9-6. Multimodal embedding models can create embeddings for multiple modalities in the same vector space.

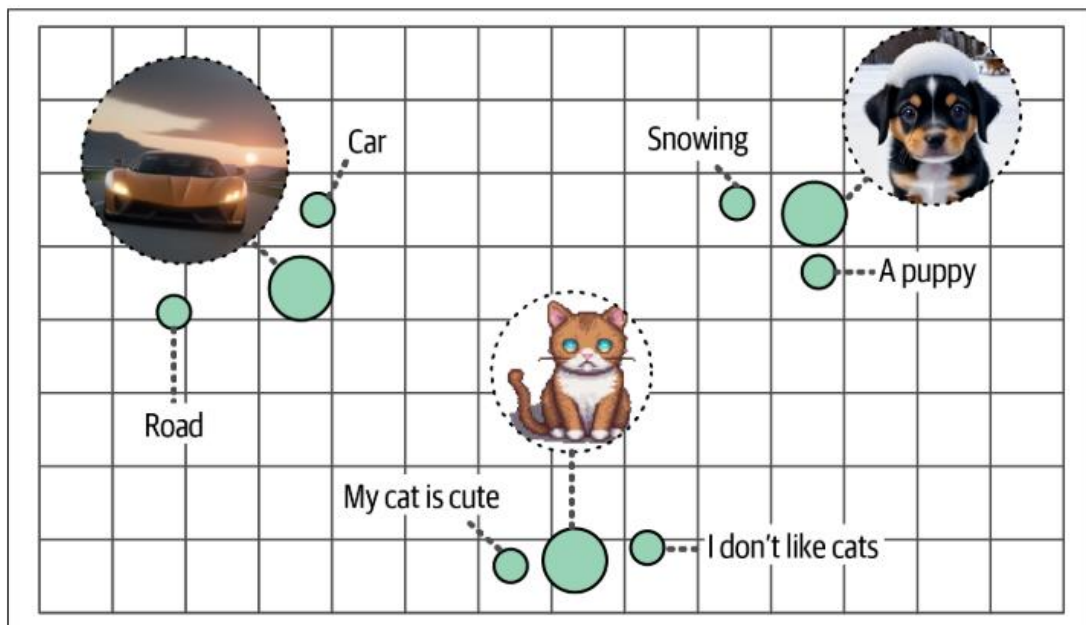


Figure 9-7. Despite having coming from different modalities, embeddings with similar meaning will be close to each other in vector space.

多模态嵌入模型有很多，但最著名和目前使用最多的模型是对比语言图像预训练 **Contrastive Language-Image Pre-training (CLIP)** 模型。

CLIP 如何生成多模态嵌入？

数据集中包含数百万张图片和对应的标题，该数据集可用于为每对图像及其标题创建两种表示。为此，CLIP 使用文本编码器来嵌入文本，使用图像编码器来嵌入图像。



Figure 9-8. The type of data that is needed to train a multimodal embedding model.

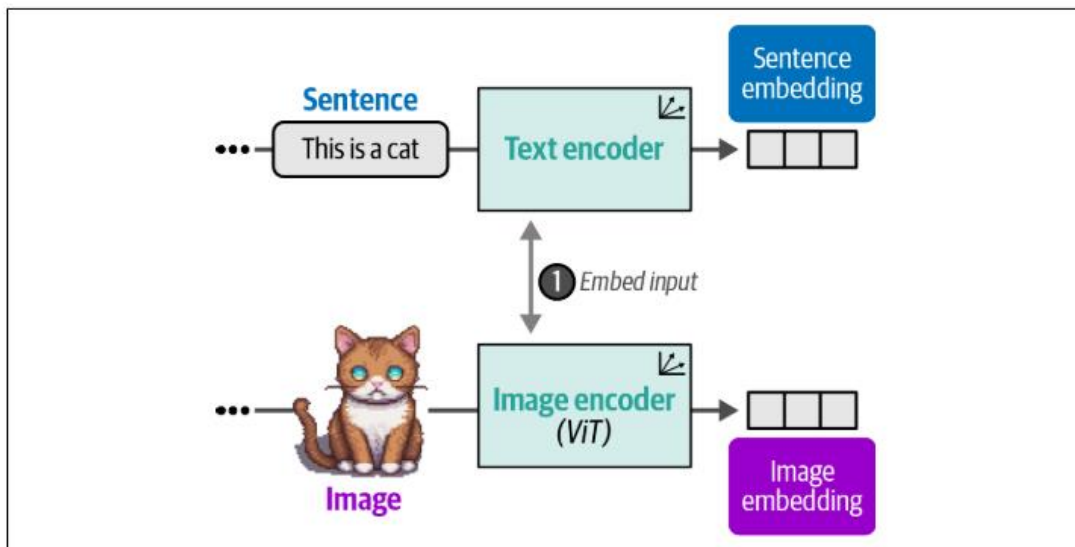


Figure 9-9. In the first step of training CLIP, both images and text are embedded using an image and text encoder, respectively.

通过余弦相似度对生成的嵌入对进行比较。当我们开始训练时，图像嵌入和文本嵌入之间的相似性会很低，因为它们还没有优化到相同的向量空间内。在训练过程中，我们对嵌入之间的相似性进行优化，并希望对于相似的图像/标题对最大化嵌入，对于不相似的图像/标题对最小化嵌入。计算出它们的相似度后，更新模型，并使用新批次的数据和更新的表示重新开始该过程（图 9-11）。这种方法被称为对比学习。

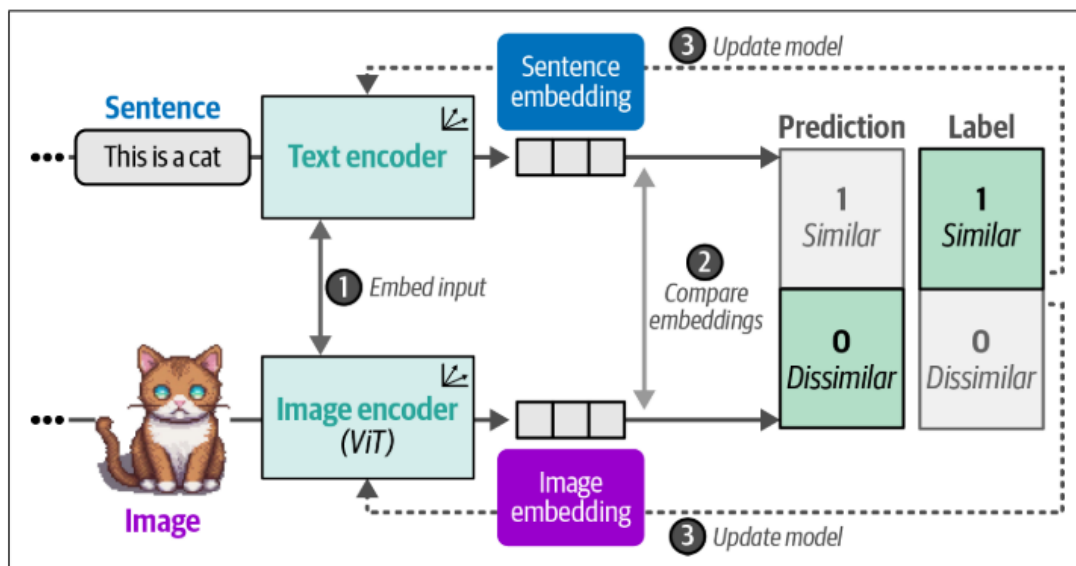


Figure 9-11. In the third step of training CLIP, the text and image encoders are updated to match what the intended similarity should be. This updates the embeddings such that they are closer in vector space if the inputs are similar.

			
A puppy playing in the snow	0.33	0.19	0.11
A pixelated image of a cute cat	0.15	0.35	0.09
A supercar on the road with the sunset in the background	0.08	0.13	0.31

下列代码示例使用来自 CLIP 的开放源代码变体的模型 OpenCLIP。

由于这张图片有一个标题，我们可以使用 OpenCLIP 为两者生成嵌入。

CLIP

```

[2] from urllib.request import urlopen
    from PIL import Image

    # Load an AI-generated image of a puppy playing in the snow
    puppy_path = "https://raw.githubusercontent.com/HandsOnLLM/Hands-On-Large-Language-Models/main/chapter09/images/puppy.png"
    image = Image.open(urlopen(puppy_path)).convert("RGB")
    caption = "a puppy playing in the snow"

```

image



为了生成嵌入，我们需要下载三个模型。

▼ Embeddings

```
from transformers import CLIPTokenizerFast, CLIPProcessor, CLIPModel

model_id = "openai/clip-vit-base-patch32"

# Load a tokenizer to preprocess the text
clip_tokenizer = CLIPTokenizerFast.from_pretrained(model_id)

# Load a processor to preprocess the images
clip_processor = CLIPProcessor.from_pretrained(model_id)

# Main model for generating text and image embeddings
model = CLIPModel.from_pretrained(model_id)
```

```
[5] # Tokenize our input
inputs = clip_tokenizer(caption, return_tensors="pt")
inputs

{'input_ids': tensor([[49406, 320, 6829, 1629, 530, 518, 2583, 49407]]), 'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 1, 1]])}

[6] # Create a text embedding
text_embedding = model.get_text_features(**inputs)
text_embedding.shape

torch.Size([1, 512])
```

```
[7] # Preprocess image
processed_image = clip_processor(
    text=None, images=image, return_tensors="pt"
)["pixel_values"]

[8] processed_image.shape

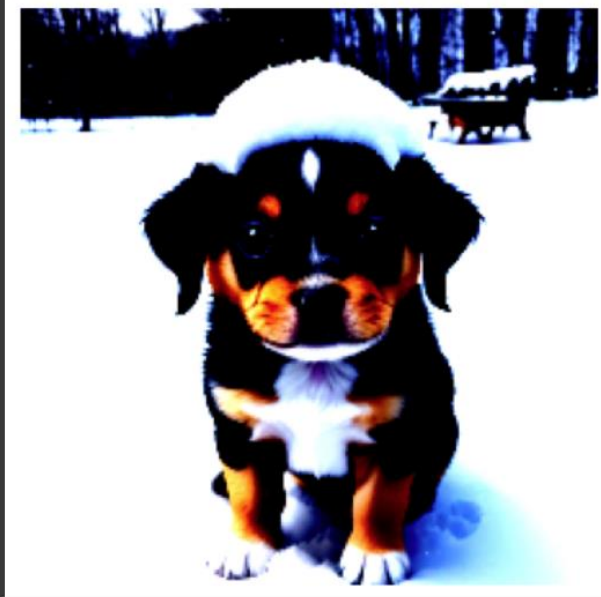
torch.Size([1, 3, 224, 224])
```

```
import torch
import numpy as np
import matplotlib.pyplot as plt

# Prepare image for visualization
img = processed_image.squeeze(0)
img = img.permute(*torch.arange(img.ndim - 1, -1, -1))
img = np.einsum("ijk->jik", img)

# Visualize preprocessed image
plt.imshow(img)
plt.axis("off")
```

WARNING:matplotlib.image:Clipping input data to the valid range f
(np.float64(-0.5), np.float64(223.5), np.float64(223.5), np.float




```
[10] # Create the image embedding
      image_embedding = model.get_image_features(processed_image)
      image_embedding.shape
```

```
⇒ torch.Size([1, 512])
```

```
[11] # Normalize the embeddings
      text_embedding /= text_embedding.norm(dim=-1, keepdim=True)
      image_embedding /= image_embedding.norm(dim=-1, keepdim=True)

      # Calculate their similarity
      text_embedding = text_embedding.detach().cpu().numpy()
      image_embedding = image_embedding.detach().cpu().numpy()
      score = np.dot(text_embedding, image_embedding.T)
      score
```

```
⇒ array([[0.33149645]], dtype=float32)
```

三、使文本生成模型多模态

对于文本生成模型，我们希望它能对某些输入图像进行推理。例如，我们可以给它一个披萨的图像，询问它包含哪些成分。可以给它一张埃菲尔铁塔的照片，问它是什么时候建的，或者它在哪里。

为了弥合图像和文本这两个领域之间的差距，已经尝试将一种形式的多模态引入现有模型。**BLIP-2: Bootstrapping Language-Image Pre-training**，用于统一视觉语言理解。BLIP-2 是一种易于使用的模块化技术，允许将视觉功能引入现有的语言模型。

从头开始创建多模态语言模型需要大量的计算能力和数据。我们必须使用数十亿个图像、文本和图像-文本对来创建这样的模型。BLIP-2 没有从头开始构建架构，而是通过构建一个名为 Querying Transformer (Q-Former) 的桥梁来弥合视觉语言的差距，该桥梁连接了一个预训练的图像编码器和一个预训练的 LLM。通过利用预训练模型，BLIP-2 只需要训练网桥，而不需要从头开始训练图像编码器和 LLM。它充分利用了已经存在的技术和模型。

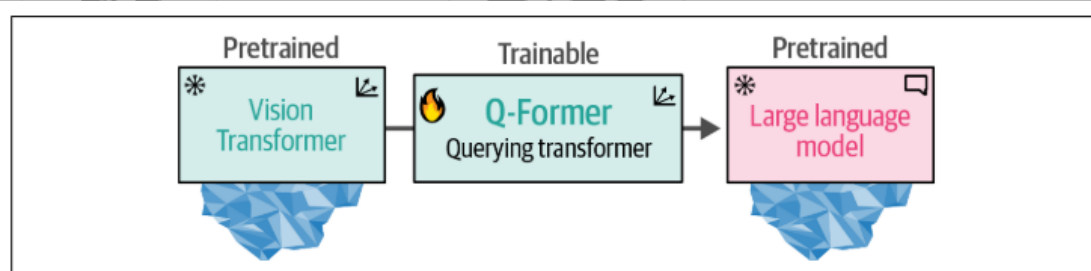


Figure 9-16. The Querying Transformer is the bridge between vision (ViT) and text (LLM) that is the only trainable component of the pipeline.

加载模型：

BLIP

```
from transformers import AutoProcessor, Blip2ForConditionalGeneration
import torch
# Load processor and main model
blip_processor = AutoProcessor.from_pretrained("Salesforce/blip2-opt-2.7b")
model = Blip2ForConditionalGeneration.from_pretrained(
    "Salesforce/blip2-opt-2.7b",
    torch_dtype=torch.float16
)

# Send the model to GPU to speed up inference
device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)
```

processor_config.json: 100%  68.0/68.0 [00:00<00:00, 7.57kB/s]

preprocessor_config.json: 100%  432/432 [00:00<00:00, 42.8kB/s]

tokenizer_config.json: 100%  882/882 [00:00<00:00, 98.6kB/s]

vocab.json:  798k/? [00:00<00:00, 33.4MB/s]

merges.txt:  456k/? [00:00<00:00, 28.8MB/s]

tokenizer.json:  3.56M/? [00:00<00:00, 91.4MB/s]

added_tokens.json: 100%  23.0/23.0 [00:00<00:00, 2.71kB/s]

special_tokens_map.json: 100%  548/548 [00:00<00:00, 65.7kB/s]

config.json:  1.03k/? [00:00<00:00, 106kB/s]

model.safetensors.index.json:  122k/? [00:00<00:00, 11.5MB/s]

Fetching 2 files: 100%  2/2 [01:54<00:00, 114.34s/it]

model-00001-of-00002.safetensors: 100%  10.0G/10.0G [01:54<00:00, 52.7MB/s]

model-00002-of-00002.safetensors: 100%  4.98G/4.98G [01:16<00:00, 26.0MB/s]

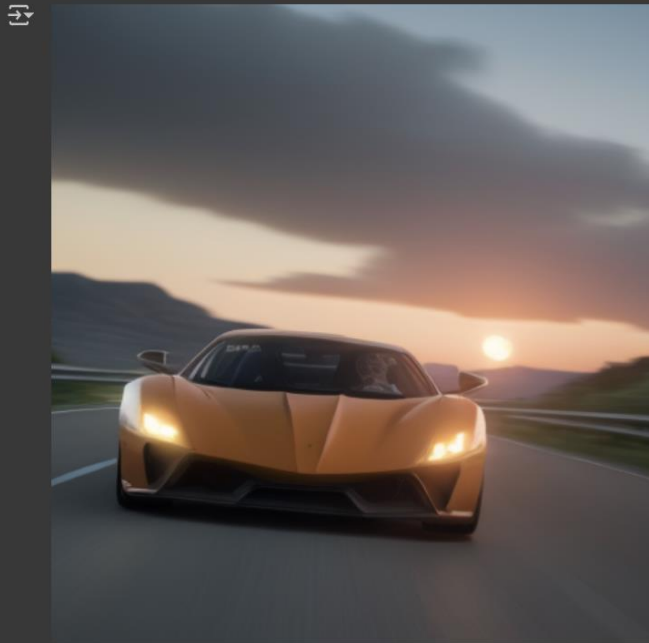
Loading checkpoint shards: 100%  2/2 [01:12<00:00, 34.04s/it]

generation_config.json: 100%  141/141 [00:00<00:00, 9.76kB/s]

下载图像:

- Preprocessing images

```
# Load image of a supercar
car_path = "https://raw.githubusercontent.com/HandsOnLLM/Hands-On-Large-Language-Models/main/chapter09/images/car.png"
image = Image.open(urlopen(car_path)).convert("RGB")
image
```



```
[ ] image.size
```

→ (492, 520)

```
[ ] # Preprocess the image
inputs = blip_processor(image, return_tensors="pt").to(device, torch.float16)
inputs["pixel_values"].shape
```

```
→ torch.Size([1, 3, 224, 224])
```

结果是一个 224×224 大小的图像。这也意味着图像的所有原始不同形状都将被处理成正方形。所以要小心输入非常宽或高的图像，因为它们可能会被扭曲。

- Preprocessing text

```
[+] blip_processor.tokenizer
```

```
<?xml><tokenizerFastName=or_path=Salesforce/blip-opt-2-7b/, vocab_size=50265, model_max_length=10000000000000009884624838656, is_fast=True, padding_side='right', truncation_side='right', special_tokens=['bos_token': '</o>', 'eos_token': '</o>', 'unk_token': '</o>', 'pad_token': '<qnd>'], clean_up_tokenization_spaces=False, added_tokens_decoder=[  
    1: AddedToken('<qnd>', rstrip=False, lstrip=False, single_word=False, normalized=True, special=True),  
    2: AddedToken('<o>', rstrip=False, lstrip=False, single_word=False, normalized=True, special=True),  
    50265: AddedToken('<image>', rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),  
]  
</tokenizer>
```

为了探索 GPT 2 Tokenizer 是如何工作的，我们可以用一个小句子来尝试一下。首先将句子转换为 token ID，然后再将它们转换回 token：


```
[ ] # Replace the space token with an underscore
text_tokens = [token.replace(" ", "_") for token in text_tokens]
text_tokens
```

```
['Her', '_vocal', 'ization', '_was', '_remarkably', '_mel', 'odic']
```

▼ Case1:Image Captioning

```
# Load an AI-generated image of a supercar
image = Image.open(urlopen(car_path)).convert("RGB")

# Convert an image into inputs and preprocess it
inputs = blip_processor(image, return_tensors="pt").to(device, torch.float16)
image
```

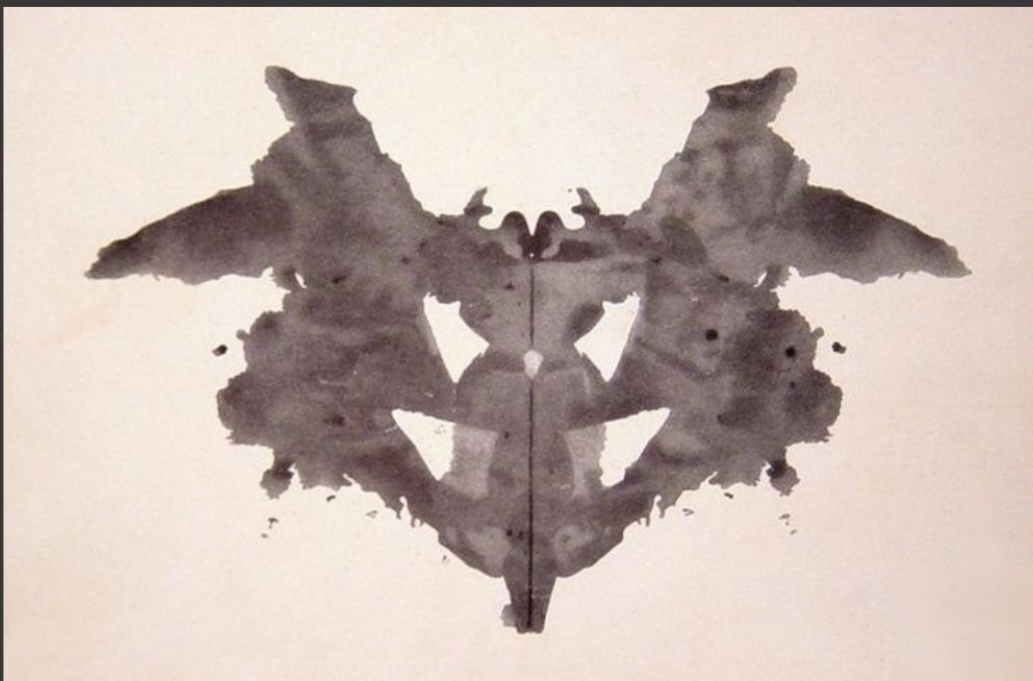



```
[ ] # Generate image ids to be passed to the decoder (LLM)
generated_ids = model.generate(**inputs, max_new_tokens=20)

# Generate text from the image ids
generated_text = blip_processor.batch_decode(
    generated_ids, skip_special_tokens=True
)
generated_text = generated_text[0].strip()
generated_text
```

⇒ 'an orange supercar driving on the road at sunset'

```
▶ # Load Rorschach image
url = "https://upload.wikimedia.org/wikipedia/commons/7/70/Rorschach_blot_01.jpg"
image = Image.open(urlopen(url)).convert("RGB")
image
```



```
[ ] # Generate caption
inputs = blip_processor(image, return_tensors="pt").to(device, torch.float16)
generated_ids = model.generate(**inputs, max_new_tokens=20)
generated_text = blip_processor.batch_decode(generated_ids, skip_special_tokens=True)
generated_text = generated_text[0].strip()
generated_text
```

⇒ 'a black and white ink drawing of a bat'

在第二个用例中，我们给模型一个图像和一个关于该特定图像的问题，让它回答。模型需要同时处理图像和问题。

Case2: Multimodal Chat-Based Prompting

```
[ ] # Load an AI-generated image of a supercar
image = Image.open(urlopen(car_path)).convert("RGB")

# Visual question answering
prompt = "Question: Write down what you see in this picture. Answer:"

# Process both the image and the prompt
inputs = blip_processor(image, text=prompt, return_tensors="pt").to(device, torch.float16)

# Generate text
generated_ids = model.generate(**inputs, max_new_tokens=30)
generated_text = blip_processor.batch_decode(
    generated_ids, skip_special_tokens=True
)
generated_text = generated_text[0].strip()
generated_text
```

'Question: Write down what you see in this picture. Answer: A sports car driving on the road at sunset'

我们可以以基于聊天的方式提出后续问题。要做到这一点，我们可以给模型我们之前的对话，包括它对我们问题的回答。然后，我们问它一个后续问题：

```
[ ] # Chat-like prompting
prompt = "Question: Write down what you see in this picture. Answer: A sportscar driving on the road at sunset. Question: What would it cost me to drive that car? Answer:"

# Generate output
inputs = blip_processor(image, text=prompt, return_tensors="pt").to(device, torch.float16)
generated_ids = model.generate(**inputs, max_new_tokens=30)
generated_text = blip_processor.batch_decode(generated_ids, skip_special_tokens=True)
generated_text = generated_text[0].strip()
generated_text
```

'Question: Write down what you see in this picture. Answer: A sportscar driving on the road at sunset. Question: What would it cost me to drive that car? Answer: \$1,000,000'

最后，我们可以通过使用 ipywidgets 创建一个交互式聊天机器人来使这个过程变得更流畅，ipywidgets 是一个针对 iPhyter 笔记本的扩展，允许我们制作交互式按钮，输入文本等：

```

▶ #通过使用ipywidgets创建一个交互式聊天机器人
from IPython.display import HTML, display
import ipywidgets as widgets

def text_eventhandler(*args):
    try:
        question = args[0]["new"]
        if question:
            args[0]["owner"].value = ""

            # 创建提示 - 仅保留最近的1-2轮对话
            if not memory:
                prompt = f"Question: {question} Answer:"
            elif len(memory) == 1:
                q1, a1 = memory[-1]
                prompt = f"Question: {q1} Answer: {a1}. Question: {question} Answer:"
            else:
                # 只保留最近两轮对话
                q1, a1 = memory[-2]
                q2, a2 = memory[-1]
                prompt = f"Question: {q1} Answer: {a1}. Question: {q2} Answer: {a2}. Question: {question} Answer:"

            #print(f"Prompt: {prompt}") # 调试输出

            # Generate text
            inputs = blip_processor(image, text=prompt, return_tensors="pt")
            inputs = inputs.to(device, torch.float16)
            generated_ids = model.generate(**inputs, max_new_tokens=100)
            generated_text = blip_processor.batch_decode(
                generated_ids,
                skip_special_tokens=True
            )
            generated_text = generated_text[0].strip()
            #generated_text = generated_text[0].strip().split("Question")[0]

            # 改进的后处理 - 移除多余的对话标记
            # 找到最后一个"Answer:"之后的内容
            if "Answer:" in generated_text:
                generated_text = generated_text.split("Answer:")[1].strip()

```

```

# 移除回答中的"Question"之后的内容
if "Question:" in generated_text:
    # 找到第一个"Question:"出现的位置
    question_index = generated_text.find("Question:")
    if question_index > 0:
        generated_text = generated_text[:question_index].strip()

# 更新记忆（但限制记忆长度）
memory.append((question, generated_text))
if len(memory) > 5: # 只保留最近的5轮对话
    memory.pop(0)

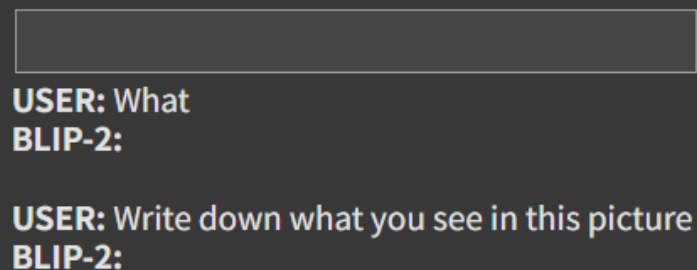
# Assign to output
output.append_display_data(HTML("<b>USER:</b> " + question))
output.append_display_data(HTML("<b>BLIP-2:</b> " + generated_text))
output.append_display_data(HTML("<br>"))
except Exception as e:
    import traceback
    error_msg = f"<b>ERROR:</b> {str(e)}<br>{traceback.format_exc()}"
    output.append_display_data(HTML(error_msg))
    output.append_display_data(HTML("<br>"))

# Prepare widgets
in_text = widgets.Text(description="You:")
in_text.continuous_update = False
in_text.observe(text_eventhandler, "value")
output = widgets.Output()
memory = []

# Display chat box
display(widgets.VBox(
    children=[output, in_text],
    layout=widgets.Layout(display="inline-flex", flex_flow="column-reverse"),
))

```

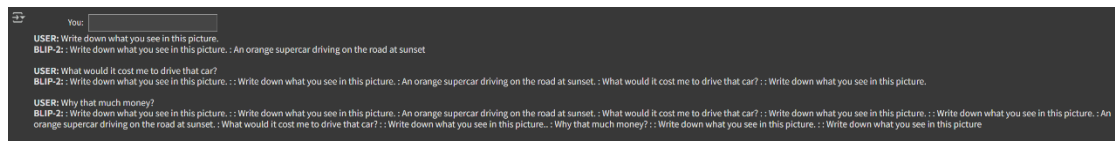
书本上的原代码没有考虑模型生成的回答还包含了 question（generated_text 格式：Question: Answer:），generated_text = generated_text[0].strip().split("Question")[0]这句代码提取了第一个元素 Question，并以 Question 为分割符，输出 Question 之前的内容，所以输出是空白。改进后的代码提取最后一个元素 Answer，而不是第一个元素 Question。



USER: What
BLIP-2:

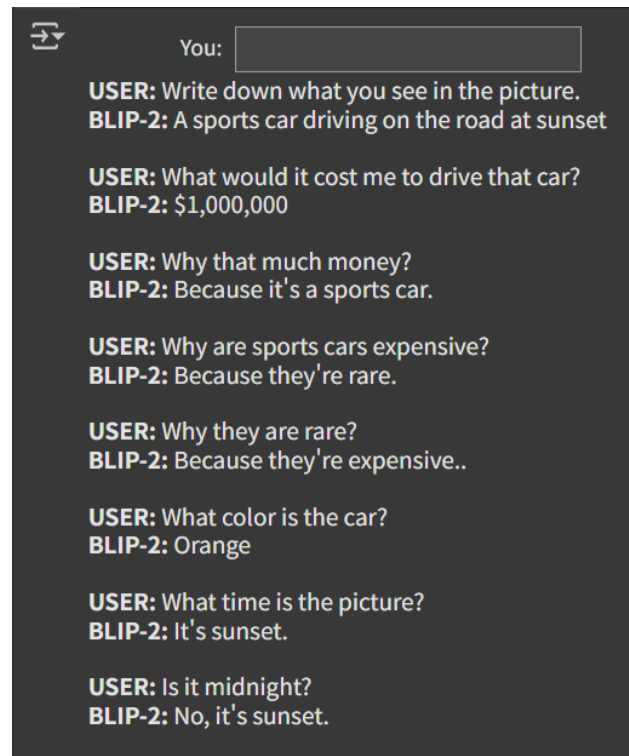
USER: Write down what you see in this picture
BLIP-2:

如果直接输出模型的 answer:



USER: Write down what you see in this picture.
BLIP-2: Write down what you see in this picture. : An orange supercar driving on the road at sunset
USER: What would it cost me to drive that car?
BLIP-2: Write down what you see in this picture. : An orange supercar driving on the road at sunset. : What would it cost me to drive that car? : Write down what you see in this picture.
USER: Why that much money?
BLIP-2: Write down what you see in this picture. : An orange supercar driving on the road at sunset. : What would it cost me to drive that car? : Write down what you see in this picture. : An orange supercar driving on the road at sunset. : What would it cost me to drive that car? : Write down what you see in this picture. : Why that much money? : Write down what you see in this picture. : Write down what you see in this picture

在修改代码时,要考虑内存问题, memory 保留五段对话, prompt 保留两段对话, 否则在回答五个以上问题时, 从第五个问题开始, 模型会生成相同的答案。



You:

USER: Write down what you see in the picture.
BLIP-2: A sports car driving on the road at sunset

USER: What would it cost me to drive that car?
BLIP-2: \$1,000,000

USER: Why that much money?
BLIP-2: Because it's a sports car.

USER: Why are sports cars expensive?
BLIP-2: Because they're rare.

USER: Why they are rare?
BLIP-2: Because they're expensive..

USER: What color is the car?
BLIP-2: Orange

USER: What time is the picture?
BLIP-2: It's sunset.

USER: Is it midnight?
BLIP-2: No, it's sunset.